

Budgetwise

Team Name: Team 1

Team Members:

Sophia Chau
Jonathan Gruber
Haru Kimura
Casandra Marquez
Marcelo Mejia
Jonathan Strong

Supervisor:

Professor Sridhar Alagar

Project Coordinator:

Adarsh Gella

**Erik Jonsson School of Engineering and Computer Science
University of Texas at Dallas**

Date: May 10, 2026

Table of contents

Executive summary

- 1. Introduction**
- 2. Project specification & management**
- 3. System architecture & high-level design**
- 4. Implementation & AI orchestration**
- 5. System performance & validation**
- 6. Lessons learned**
- 7. Conclusion & future work**

References

Acknowledgements

Executive summary

Budgetwise is a student-focused budgeting web application designed to make financial planning more approachable, actionable, and sustainable. The project was motivated by a common challenge among college students: managing tuition, rent, groceries, and other recurring expenses without established budgeting habits or financial training. Team 1 developed Budgetwise to reduce this friction by combining expense tracking, monthly budget planning, dashboard visualizations, and AI-assisted guidance into a single responsive platform.

The project achieved most of its primary objectives. Users can track expenses and income, create and customize monthly budgets by category, and monitor progress through visual dashboard elements such as pie and bar charts. The application also provides smart spending warnings when users approach category limits and includes AI-assisted recommendations to improve spending behavior. In addition, the system supports a broader audience beyond students, extending the product's practical value.

Technically, Budgetwise was implemented as a full-stack web system using React [1] and Next.js [2] with TypeScript [3], Tailwind CSS [4], and charting libraries on the frontend, and Express [5], Prisma [6], PostgreSQL [7], and supporting middleware on the backend. The platform integrates external services for key capabilities: Groq-based LLM endpoints [8] for structured budgeting insights, Plaid [9] for account-linking and transaction import workflows, and email-based password reset support using configurable mail-server infrastructure. During development, the team also adapted integration strategies in response to real deployment constraints, including a shift from Gemini [10] to Groq due to token limits and practical email relay adjustments for cloud deployment.

Deployment followed a containerized architecture using Docker [11] and Docker Compose [12] for local development, with Render [13] hosting production frontend and backend services and managing PostgreSQL in the cloud. This architecture improved consistency across environments and enabled scalable iteration through GitHub-based version control and CI/CD workflows.

Scalability for Budgetwise was considered in the contexts of growing the app with future additions as well as the app's performance under increasing users and workload. To achieve this, the team made constant improvements to the manner in which data was exchanged between the backend, frontend, and database. Moreover, HTTP request testing and documentation was performed on API endpoints to measure the effect these changes had on resource-intensive areas.

Beyond technical delivery, the project strengthened team capabilities in AI-assisted software engineering, specification-driven development, and agile collaboration. The final product delivers a functional, extensible budgeting platform and establishes a strong foundation for future enhancements such as richer goal tracking, full Plaid production integration, and more advanced long-term AI financial projections.

1 Introduction

1.1 Motivation & problem statement

Since our group is composed of college students, we thought that it would be both interesting and productive to attempt to address a problem that many of our fellow college students face. One such problem that immediately comes to mind is that of finances. The life of a college student can involve a variety of expenses, from the purely school-related such as tuition, textbooks, and school supplies, to the more mundane such as rent and groceries. For a student who has for most of their life had finances managed by others and is only just beginning to experience some degree of independence, it can be daunting to attempt to navigate this array of expenses. Many students lack any financial education or structures that would enable them to efficiently manage their finances.

1.2 Objectives, scope, and goals achieved

Key objectives

- Build a fully functional budgeting web application that allows users to track expenses, set budgets, and monitor financial goals
- Reduce the complexity and intimidation associated with starting a budget
- Encourage healthier financial behaviors through forecasting and proactive insights
- Deliver a scalable architecture that can support future features and expanded audiences

Scope

- Must-haves
 - User expense tracking with categorized spending displayed as percentage breakdowns (pie chart)
 - Ability to set monthly budgets and category limits
 - Dashboard showing spending progress and remaining budget per category
 - End of month spending forecast based on current expenses, budget data, and past expenses using AI
 - Responsive web interface built with React [1], backend APIs for data storage and retrieval using Next.js [2]
 - AI API integration used for lightweight categorization or forecasting support, where structured prompts are sent from the backend and responses are returned in a predefined schema for frontend display
 - Goal tracking for student loans, emergency funds, and investing
- Nice-to-haves
 - Smart spending warnings when users approach category limits
 - AI assisted budget suggestions based on historical data
- Stretch goals
 - Bank integrations with automated transaction syncing
 - Advanced AI driven insights, alerts, and recommendations
 - Long term financial projections beyond a single month
 - Support for non student users and family or shared budgets

Goals achieved

- User expense tracking with categorized spending displayed as percentage breakdowns in the form of a pie chart.
- Ability to set monthly budgets and category limits

- Dashboard showing spending progress and remaining budget per category in the form of a bar chart.
- End of month spending forecast in the form of recommended actions based on current expenses, budget data, and past expenses using AI
- Responsive web interface built with React [1], backend APIs for data storage and retrieval using Next.js [2]
- AI API integration used for lightweight categorization or forecasting support, where structured prompts are sent from the backend and responses are returned in a predefined schema for frontend display
- Smart spending warning in the dashboard as user approach category limits
- AI assisted budget suggestion based on historical data
- Support for non student users

1.3 Proposed solution

A core instrument for managing one's finances is the ability to maintain and use a budget. To this end, our group has designed a budgeting application aimed primarily at college students. The application simplifies the budgeting process by centralizing expense tracking, goal planning, and financial forecasting in one location, thereby also reducing the friction that prevents students from starting and maintaining a budget. Our application aims to help cultivate and support good long-term financial habits by helping users visualize their spending patterns, plan monthly budgets, and anticipate end-of-month outcomes to prevent overspending. Moreover, while the primary audience of our application is certainly college students, we have intentionally designed the application to be usable by anyone with budgeting needs.

1.4 Key highlights

1.4.1 AI integration

This application integrates AI through authenticated backend endpoints that collect each user's monthly expense and budget data from Prisma [6], transform it into category-level summaries, and send that context to Groq Chat Completions [8] to generate structured finance guidance. On the frontend, the dashboard's recommendations component automatically calls the AI endpoints for (1) three actionable insight cards ('reduce', 'keepDoing', 'spendMore') and (2) an AI-vs-current spending comparison chart, while budget-creator flows can request AI-adjusted category percentage suggestions to refine allocations.

Originally, AI integration was done through Gemini's free tier [10]. However, we ran into multiple problems with token limits. To fix this, we began using Groq, which offers multiple free models with higher token and request limits.

1.4.2 Mail servers

In order to allow the user to reset their password, the application must send an email message containing a password-reset link to the email address of the user. In order to do this, the application must connect to a mail server and submit to it the email message; the mail server then handles the task of delivering the message to the user's mailbox. The backend is responsible for connecting to the mail server, and how it connects to the mail server is configurable through an environment variable named "MAIL_SERVER_URL"; by default, the backend attempts to connect to a mail server upon the same machine that is listening upon port 587, which is one of the standard ports for submitting an email message to a mail server [14].

By default, for local deployment via Docker Compose [12], we use Postfix [15], a free and open-source mail-server program, to run a self-hosted mail server. While the backend is able to successfully connect to this mail server and submit the reset-password email message for delivery to the user, in our testing, the application's mail server was unable to connect to the external mail server corresponding to the email address of the user in order to actually deliver the reset-password email message. According to Postfix's logs, the attempts to connect to the external mail server would consistently time out, suggesting not that the connection requests were being refused but might not even be reaching the external mail server. The standard port upon which a mail server listens for connection requests from other mail servers is port 25 [14], so we then tried ourselves to connect to the external mail server and obtained similar connection time-out errors. Indeed, in our tests, connecting to *any* external host upon port 25 resulted in the same errors. Our hypothesis is that some firewall, probably belonging to a local ISP, along the route from our local machines to the external mail servers is blocking outbound connections to port 25, severely limiting our ability to test the self-hosted mail server upon our local machines.

The platform we chose for deploying the application on the web, Render [13], does not seem to support deployment of a mail server, at least with its free tier. As a result, for web deployment, we were forced to improvise by relaying the application's email messages via a throwaway Gmail account, which was relatively simple to configure by appropriately setting the "MAIL_SERVER_URL" environment variable. In particular, the backend connects to port 465, the standard port for submitting an email message to a mail server directly over TLS [14], upon Gmail's mail server (at smtp.gmail.com [16]), authenticates itself into the throwaway Gmail account, and then submits the email message to Gmail's mail server through the connection. Gmail's mail server then handles the task of actually delivering the email message to the mailbox of the user. This is not an ideal solution, but it is the best solution that we could devise that did not require us to spend any of our money.

1.4.3 Plaid integration

Plaid integration [9] in this project works like a secure import pipeline: after a logged-in user connects a bank account through Plaid Link [17], the backend creates a link token, receives a short-lived public token, exchanges it for a long-lived Plaid access token, saves the linked account details, then immediately syncs transactions; each synced transaction is mapped into the application's own expense or income format (including category mapping and transfer filtering), upserted so duplicates are avoided, and tracked with a cursor so later syncs only pull new or changed transactions, while removed Plaid transactions are also removed from local records to keep the dashboard accurate.

We decided to use Plaid because it was one of the only banking integration tools that offers a free demo version. We integrated Plaid using their Plaid Sandbox feature [18], which gives limited functionality. Currently, a user can open Plaid but not link any accounts due to the limitations of Sandbox. We can also demo importing pre-existing example data through Sandbox.

1.4.4 Deployment

The application is deployed using a containerized architecture to ensure consistency between development and production environments. Docker [11] is used to package the frontend, backend, and database into isolated containers, allowing each service to run reliably regardless of the underlying system. Docker Compose [12] is utilized during local development to orchestrate these multiple services, enabling the entire application stack to be launched with a single command. For cloud deployment, the application is hosted on Render [13], which provides managed infrastructure for both the frontend and backend services. Render handles service deployment, environment variable configuration, and automatic builds, allowing the team to focus on application functionality rather than infrastructure management. The backend is deployed as a web service, while the frontend is deployed as a separate service, both connected

through defined environment variables. The production database is PostgreSQL [7]. During local development, PostgreSQL runs within a Docker container, while in deployment, a managed PostgreSQL instance is used to ensure persistence and reliability. Additionally, GitHub is used to manage version control and support CI/CD pipelines. Through GitHub Actions, the project can automate build and deployment workflows, ensuring that updates to the codebase can be tested and deployed efficiently. This deployment setup provides a balance between simplicity and scalability, allowing the application to be easily developed, tested, and extended in future iterations.

2 Project specification & management

2.1 Project specification

Project specification:

https://docs.google.com/document/d/1f8t0OkfDzziXCPE1h98CdHhTq_KgU0DuoDzA2j8-bNU/edit?usp=sharing

This project specifications document outlines the product overview, tech stack, file structure, phase requirements, UI/UX guidelines, application layout, non-functional requirements, and AI instructions and constraints. It was used to guide AI agents when generating code to ensure that all AI-generated code adhered to key requirements and helped with compatibility.

2.2 Maintaining the “single source of truth”

The project specs were maintained primarily on Google Docs and were uploaded to GitHub every time the document was updated to ensure all team members have access to the latest version. The document was maintained primarily by Sophia, and was intermittently manually reviewed to ensure that it reflected the current state of the project.

2.3 Key milestones

2.3.1 Technical Specs

The technical specs were first started on February 24, 2026, during the first few weeks of the project. They were iteratively modified given feedback from the professor and TA. Key modifications include updating the project file tree (3/4), adding UI/UX guidelines (3/5), and adding the color scheme (3/10).

2.3.2 First Functional MVP

The first functional MVP was created during week 3 (2/20-2/27). The first iteration included a login page, database for storing information, a calendar page for expenses, and a dashboard to view expenses. The program would come to have full functionality that also includes password reset through an automated email, AI features and recommendations, Plaid integration [9], and a full deployment.

2.3.3 Key Deployment Stages

The application was deployed using Docker [11] and Render [13]. The first full deployment happened during week 8 (04/04-04/10), and included a deployment of the frontend, backend, and database services on Render. Then, the team began a way to automate deployment while also using a CI/CD pipeline via GitHub during week 9 (04/11-04/16). Now, the program redeploys every time it is updated on the GitHub main branch via a pull request.

2.4 Team roles & leader rotation

Duration	Team Leader
----------	-------------

02/01–02/14	Sophia Chau
02/15–02/28	Jonathan Gruber
03/01–03/14	Casandra Marquez
03/15–04/04	Marcelo Mejia
04/05–04/18	Jonathan Strong
04/19-05/01	Haru Kimura

Role	Team Member	Responsibilities
Backend developer	Sophia Chau	Backend development
Backend developer	Jonathan Gruber	Backend development
Frontend developer	Haru Kimura	Frontend development
Full-stack developer	Casandra Marquez	Full-stack development
Frontend developer	Marcelo Mejia	Frontend development
Frontend developer	Jonathan Strong	Frontend development

3 System architecture & high-level design

3.1 System overview

The primary components of the application are the frontend server, the backend server, the backend database, the mail server, and the Plaid API server [19]. The frontend server is the web server with which the user directly interacts in order to access the application, listening for web requests from the user's web browser and presenting its responses to the user via the application's user interface. In order to fulfill the user's web requests, the frontend server invokes the backend server's API by sending web requests to the backend server; the backend server listens for those requests and responds appropriately to the frontend server. Most of the API endpoints attached to the backend server require the server to access the application's data, so the backend server connects to the backend database, a relational database implemented via PostgreSQL [7] that stores all the application's data. Some of the backend server's API endpoints also require the application to send email messages or access the Plaid API, so the backend server also submits email messages to a mail server and sends requests to a Plaid API server.

The backend server provides the following API endpoints:

- GET /api/health
 - Checks that the backend server is running and reachable from the frontend server
- POST /api/auth/register
 - Registers a new user account
 - Returns an authentication access token, an authentication refresh token, and the account ID, email address, and name of the user
- POST /api/auth/login
 - Logs-in an existing user account
 - Returns an authentication access token, an authentication refresh token, and the account ID, email address, and name of the user
- POST /api/auth/refresh
 - Refreshes the authentication tokens of the user
 - Returns an authentication access token, an authentication refresh token, and the account ID, email address, and name of the user
- POST /api/auth/forgot-password
 - Creates a reset-password request for an existing user account
 - Directs the backend server to submit a reset-password email message to the mail server for delivery to the mailbox identified by the email address of the user account in question
 - The reset-password email message contains a URL for a reset-password page for the user to reset their password
- POST /api/auth/reset-password
 - Resets the password of an existing user account
- GET /api/auth/me
 - Deprecated
 - Returns the account ID, email address, name, and account creation time of the user
- GET /api/profile/me
 - Returns the account ID, email address, name, and account creation time of the user
- PUT /api/profile/me
 - Deprecated
 - Modifies the name or email address of the user
 - Returns the account ID, (updated) email address, (updated) name, and account creation time of the user
- PUT /api/profile/me/password

- Deprecated
 - Modifies the account password of the user
- DELETE /api/profile/me
 - Deprecated
 - Deletes the account of the user
- GET /api/expenses
 - Query parameters:
 - *from* (optional): YYYY-MM, where YYYY is a year number and MM a month number
 - *to* (optional): identical format to *from*
 - Returns the user's financial transactions from between the month identified by the parameter *from* (inclusive) and the month identified by the parameter *to* (inclusive)
 - If the parameter *from* is omitted, then there is no lower bound upon the dates of the transactions to be retrieved
 - If the parameter *to* is omitted, then there is no upper bound upon the dates of the transactions to be retrieved
- POST /api/expenses
 - Creates and returns a new financial transaction by the user
- PUT /api/expenses/*id*
 - Modifies the user's financial transaction identified by *id*, returning the modified transaction
- DELETE /api/expenses/*id*
 - Deletes the user's financial transaction identified by *id*
- GET /api/budgets
 - Query parameters:
 - *year* (optional): year number
 - *month* (optional): month number
 - Returns the user's budget for the month identified by the parameters *month* and *year*
 - If either *year* or *month* is missing, then the returned budget is the one for the current month and year
- POST /api/budgets
 - Creates a new budget for the user or modifies an existing one
- GET /api/dashboard
 - Query parameters:
 - *year* (optional): year number
 - *month* (optional): month number
 - Returns various information to be displayed upon the user's dashboard, namely information about the user's budget, income, and expenses for the month identified by the parameters *month* and *year*, or the current month and year if either parameter is missing
- GET /api/ai/dashboard-insights
 - Query parameters:
 - *year* (optional): year number
 - *month* (optional): month number
 - Returns LLM-generated budgeting recommendations for the user for the month identified by the parameters *month* and *year*
 - The parameters year and month default to the number of the current year and month, respectively
- POST /api/ai/budget-suggestions
 - Returns LLM-generated budget for the user
- GET /api/ai/budget-comparison
 - Query parameters:

- *year* (optional): year number
 - *month* (optional): month number
- Returns a comparison, of the user's budget for the month identified by the parameters month and year, to an LLM generated budget for the same month
- The parameters year and month default to the number of the current year and month, respectively
- PATCH /api/settings/profile
 - Modify the user's profile information, returning the modified profile information
- DELETE /api/settings/account
 - Delete the user's account
- POST /api/plaid/link-token
 - Creates a Plaid Link public token [17] and returns the token and its expiration time
- POST /api/plaid/exchange-public-token
 - Uses the Plaid API to link a bank account to the user's application account and to import financial transactions from the linked bank account into the user's application account
 - Returns information about the linked bank account and a summary of the financial transactions importation
- POST /api/plaid/demo-import
 - Uses the Plaid API and Plaid Sandbox [18] to link a mock bank account to the user's application account and to import the mock financial transactions from the mock bank account into the user's application account
 - Returns information about the linked mock bank account and a summary of the mock financial transactions importation
- POST /api/plaid/sync
 - Uses the Plaid API to import financial transactions from a bank account that is linked to the user's application account into said account
 - Returns a summary of the financial transactions importation
- GET /api/plaid/accounts
 - Returns the bank accounts linked to the user's application account
- GET /api/plaid/sandbox-config
 - Returns configuration for using Plaid Sandbox as mentioned above

3.2 Architecture diagram

We supply links to the architecture diagram instead of embedding a picture of it in this document, otherwise the details of the diagram would be too difficult to discern.

ODG file: <https://github.com/Casi1227/CS4485-Alagar-Team1/blob/main/Architecture%20Diagram.odg>

PDF file: <https://github.com/Casi1227/CS4485-Alagar-Team1/blob/main/Architecture%20Diagram.pdf>

3.3 Key architectural decisions and trade-offs

3.3.1 Frontend application framework

Two options that we considered for the application framework for the frontend were Spring Boot [20] and Next.js [2]. We initially intended to use Spring Boot because some of us had prior experience using Spring Boot, but, because we also intended to use React [1] for the frontend, and because Next.js is specifically designed for React-based applications, we ultimately decided to use Next.js.

3.3.2 Database management system

The choice between which database management system to use for the backend database was essentially between SQLite [21] and PostgreSQL [7]. SQLite would have been simpler and easier to configure, whereas PostgreSQL would have been more performant but more difficult to configure. We initially used SQLite but, once the configuration issues with PostgreSQL had been worked out, we switched to PostgreSQL.

3.3.3 Mail server hosting

We had the choice between relaying email messages through an external mailbox and self-hosting the mail server. Relaying through an external mailbox would have been simpler in theory but would not necessarily allow us to choose the email domain from which to send email messages. Self-hosting the mail server would in theory allow us to choose the email domain from which to send email messages but would require us to properly configure the mail server. Additionally, in our testing, the self-hosted mail server was not able to connect to external mail servers (see the discussion in Section 1.4.2). Ultimately, we chose to self-host the mail server for local deployment via Docker Compose [12] but use an external mailbox (specifically, a throwaway Gmail account) for deployment via Render [13] so that email functionality would be fully operational for the Render deployment.

3.3.4 Self-hosted mail server package

There are a variety of free and open-source software packages that provide mail servers, those of which we considered were Exim [22], OpenSMTPD [23], and Postfix [15]. OpenSMTPD seemed to be the simplest program and the easiest to configure and was the only one with which any of us (namely, Jonathan G.) had any prior experience. Nonetheless, we ultimately chose Postfix because it also seemed relatively easy to configure but also much more versatile than OpenSMTPD.

3.3.5 Scalability

We started scalability testing by measuring the performance metrics in both the local Docker [11] and deployed versions of the app, using Autocannon [24] for the local Docker environment and Locust [25] for the deployed environment. Both tools were used to simulate multiple virtual users sending concurrent requests to API endpoints to evaluate scalability under load.

One of the first general changes made to improve performance was relieving database pressure. Specifically, this included pagination to return information in “pages” rather than large rows of data. This was also included with cursor support to avoid skipping or duplicating rows when data changes between requests.

Another one of the changes introduced was horizontal scaling, added through the use of Nginx [26] to create backend replicas where users could be routed to. This relieves the traffic in the login by sending users to one of the replicas if the main backend is experiencing too much load. JWT authorization [27] is also used in conjunction with this to relieve pressure, since users with valid access/refresh tokens are not rerouted to the resource-intensive login page.

As the amount of users increases, so will the amount of tokens used for our AI recommendations and chart. In order to help reduce the token costs we implemented browser caching for the responses. Upon initial loading of the dashboard it receives the AI responses and stores them. Then as you navigate and add/remove expenses and incomes it continues to display the previously stored information. However on

a refresh a new response will be requested with the most up to date information and the new response will overwrite the previous one.

4 Implementation & AI orchestration

4.1 Technologies used

4.1.1 Frontend stack

- **Next.js [2]:** The web-application framework for the UI layer. It powers the routes in the frontend application directory and uses the App Router pattern for pages, layouts, and client components.
- **React [1]:** The component model for screens like the dashboard. It handles state, effects, and interactive rendering.
- **TypeScript [3]:** Used across the frontend for typed props, API responses, and component state. It reduces integration errors between UI and backend contracts.
- **Tailwind CSS [4]:** The primary styling system. It drives most of the dashboard layout and visual styling through utility classes and theme tokens.
- **React Context and browser storage [1]:** Used for auth/session management, theme mode, and user data. This is where login state is managed.
- **Recharts [28]:** Used for the financial visualizations, including spending and budget comparison charts. Its job is to render charts from API data.
- **Lucide React [29]:** The icon set used throughout the UI. It provides lightweight SVG icons with a consistent look.
- **Plaid Link React [30]:** The frontend bank-linking integration. It connects the user interface to Plaid when a user wants to link a financial account.
- **Radix UI [31]:** The frontend component primitives used in the shared UI layer. These give accessible building blocks for dialogs, menus, selects, and similar controls.
- **MUI [32]:** Included in the frontend dependency set for richer component options. The current dashboard code is mostly Tailwind-driven, but MUI is available if needed.
- **react-hook-form [33] and Zod [34]:** Used for form handling and validation in the frontend forms and shared form components. They keep login, register, and settings forms consistent and validated before submission.

4.1.2 Backend stack

- **Express [5]:** The backend HTTP server framework. It wires the API routes and serves as the REST API layer the frontend talks to.
- **Prisma [6]:** The ORM and database client. It defines the data models and is used throughout the backend routes and services to read and write users, expenses, budgets, Plaid imports, and reset tokens.
- **PostgreSQL [7]:** The database itself. It stores the application's core financial data and user accounts.
- **TypeScript [3]:** Used on the backend too, which keeps route handlers, service functions, and Prisma access typed end-to-end.
- **Zod [34]:** The request-validation layer for backend inputs. It validates login/register payloads, budget requests, Plaid sync payloads, and AI request schemas before the handlers run.
- **Nginx [26]:** A reverse proxy and load balancer that sits in front of the Express API when running scaled backends; it exposes a single host port and distributes requests across backend replicas.
- **JWT [27]:** Used for authentication tokens. The backend signs tokens for login/register, and the frontend sends them back on authenticated requests.
- **bcryptjs [35]:** Used to hash and verify passwords in the auth routes. This keeps password storage secure instead of storing plaintext.

- **Nodemailer [36]:** Used for password-reset email delivery in the auth flow. It sends reset links from the backend using the configured mail server.
- **Postfix [15]:** A free and open-source mail server package used for the self-hosted mail server for local deployment via Docker Compose.
- **Plaid SDK [9]:** The bank-connection and transaction-import integration. It powers link-token creation, token exchange, account lookup, and transaction syncing.
- **Groq SDK [8]:** Used for AI-generated budget recommendations. This is what produces the dashboard's recommendation text.
- **xlsx:** Used for spreadsheet-based mock-data import. The seed/import script loads transaction data from the workbook in the backend mock-data workflow.
- **dotenv [37]:** Loads environment variables before the server starts, so secrets and URLs come from configuration rather than hardcoded values.
- **cors [38], helmet [39], morgan [40]:** Express middleware for cross-origin access, security headers, and request logging.

4.1.3 Platform and docs

- **Docker Compose [12]:** Defines the local full-stack environment, including the Postgres database, the backend server, the frontend server, the mail server, and the mock-data seed job.
- **Render [13]:** Hosts cloud deployment of application, including frontend server, backend server, and PostgreSQL database, provides managed infrastructure for both the frontend and backend services, and handles service deployment, environment variable configuration, and automatic builds.
- **OpenAPI [41]:** The API contract documents the backend endpoints and the expected request and response shapes.
- **Figma [42]:** The frontend originated from a Figma design export. It is the design source rather than a runtime dependency.

4.1.4 Where each piece fits

- **UI layer:** Next.js, React, TypeScript, Tailwind CSS, Lucide React, Recharts, Radix UI, MUI, react-hook-form, Zod
- **Server/API layer:** Express, TypeScript, Zod, Nginx, JWT, bcryptjs, Nodemailer, Postfix, Plaid SDK, Groq SDK, xlsx, dotenv
- **Data layer:** Prisma, PostgreSQL
- **Dev/runtime tooling:** Render, Docker Compose, OpenAPI, cors, helmet, morgan

4.2 AI build strategy

The team used a combination of the project specifications and engineered prompts to build the system. Some members of the team used Cursor [43], and others used a combination of VSCode [44] and GitHub Copilot [45]. The team members would attach the project specs to a prompt/AI agent and then create a prompt around what feature they wanted to develop. No prompt library was used during this project.

4.3 Human-in-the-loop interventions

There were a couple of bugs introduced by the AI model. For example, when trying to modify the categories/percentages on the budget creator, the user would not be able to modify certain values or increase percentages for no obvious reason. The AI also would create prompts for the AI features that used too many tokens for the use case, and so those prompts had to be optimized. The AI would also often

introduce typing errors when programming, but it was often able to catch those errors by itself and correct them dynamically.

4.4 Codebase

GitHub: <https://github.com/Casi1227/CS4485-Alagar-Team1>

Readme: <https://github.com/Casi1227/CS4485-Alagar-Team1/blob/main/budgetwise/README.md>

Project folder:

https://drive.google.com/drive/folders/1R4zI69DtAqZ4fkZIkBxWSll0CczN3df?usp=drive_link

Render: <https://budgetwise-frontend-q4g8.onrender.com/>

5 System performance & validation

5.1 Testing and validation

Throughout development, the team continuously tested both individual features and full system integration to ensure the application functioned correctly across different environments. Since Budgetwise is a full-stack application with frontend, backend, database, AI, and third-party integrations, testing focused heavily on validating communication between systems and ensuring stable error handling. Manual testing was performed for core functionality such as account registration, login, password reset, expense and income tracking, budget generation, dashboard updates, and AI-generated recommendations. Authentication-related flows, especially forgot-password and reset-password functionality, required additional validation to ensure reset links, token expiration, and frontend error handling behaved correctly. Backend API endpoints were tested through direct HTTP requests and integration workflows to verify authentication, request validation, and database operations. Testing also ensured that expenses, budgets, Plaid-linked accounts [9], and AI-generated recommendations were correctly stored and retrieved through Prisma [6] and PostgreSQL [7]. AI-related functionality was validated through repeated prompt and response testing to reduce malformed outputs, excessive token usage, and inconsistent formatting. Frontend AI components were also tested to ensure proper handling of loading states, failed requests, and cached responses. Finally, deployment and scalability validation was performed in both local Docker environments [11] and the deployed Render environment [13]. Concurrent request testing tools such as Autocannon [24] and Locust [25] were used to identify resource-intensive endpoints and evaluate performance under higher user loads.

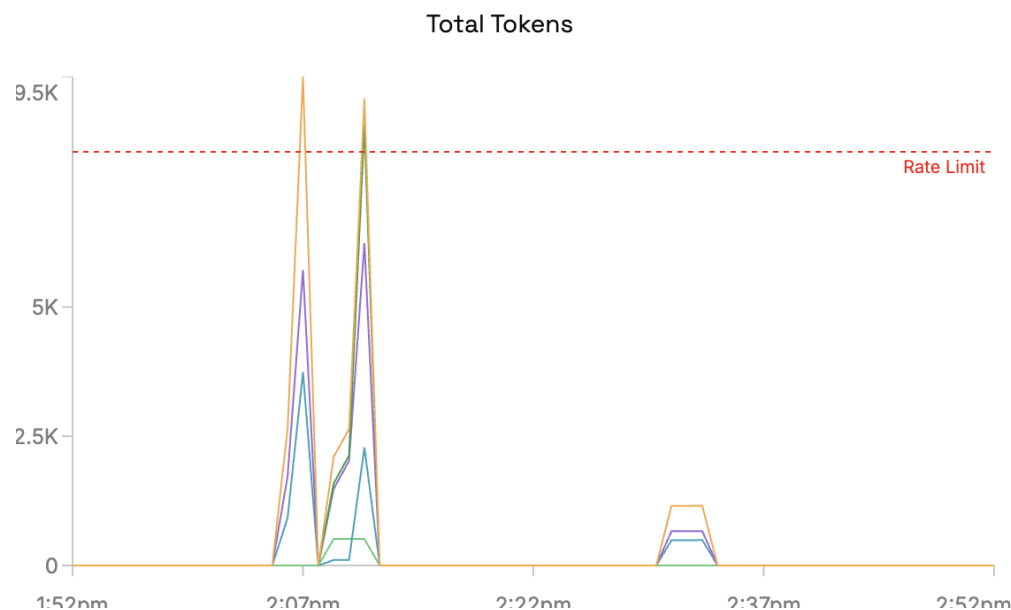
5.2 Technical performance metrics

Benchmarks									
Endpoint / Flow	Concurrency	Duration	Target p95	Observed p95	Observed Throughput	Target TR	Observed TR	Target ER	Observed ER
GET /api/expenses	50 VU	5 min	<= 500 ms	1049 ms	49.58 req/s	< 1%	0.02%	< 1%	0.02%
GET /api/expenses	100 VU	5 min	<= 500 ms	2038 ms	48.82 req/s	< 1%	6%	< 1%	6%
GET /api/expenses	250 VU	5 min	<= 900 ms	3857 ms	48.24 req/s	< 3%	17.30%	< 3%	17.30%
POST /api/expenses	50 VU	5 min	<= 700 ms	64 ms	914.72 req/s	< 1%	0.00%	< 1%	0.00%
POST /api/expenses	100 VU	5 min	<= 1000 ms	170 ms	885.87 req/s	< 2%	0%	< 2%	0%
POST /api/expenses	250 VU	5 min	<= 1800 ms	381 ms	900.18 req/s	< 5%	0.02%	< 5%	0.02%
GET /api/dashboard	50 VU	5 min	<= 900 ms	110 ms	564.06 req/s	< 2%	0%	< 2%	0%
GET /api/dashboard	100 VU	5 min	<= 1200 ms	260 ms	555.74 req/s	< 3%	0%	< 3%	0%
GET /api/dashboard	250 VU	5 min	<= 1800 ms	562 ms	549.41 req/s	< 5%	0%	< 5%	0%
POST /api/auth/login	50 VU	3 min	<= 700 ms	6498 ms	3.92 req/s	< 1%	39.10%	< 1%	39.10%
POST /api/auth/login	100 VU	3 min	<= 1000 ms	N/A	0 req/s	< 2%	100%	< 2%	100%
POST /api/auth/login	250 VU	3 min	<= 1800 ms	N/A	0 req/s	< 5%	100%	< 5%	100%
GET /api/health	50 VU	2 min	<= 100ms	20 ms	3879.35 req/s	< 0.1%	0%	< 0.1%	0%
GET /api/health	100 VU	2 min	<= 120 ms	32 ms	3957.21 req/s	< 0.1%	0%	< 0.1%	0%
GET /api/health	250 VU	2 min	<= 200 ms	71 ms	3999.15 req/s	< 0.5%	0%	< 0.5%	0%

Post Change										
Endpoint / Flow	Concurrency	Duration	Target p95	Observed p95	Observed Throughput	Target Tin	Observed Tin	Target Err	Observed Err	Backend Re
GET /api/expenses	50 VU	5 min	<= 500 ms	173 ms	350.12 req/s	< 1%	0%	< 1%	0%	N/A
GET /api/expenses	100 VU	5 min	<= 500 ms	390 ms	357.67 req/s	< 1%	0%	< 1%	0%	N/A
GET /api/expenses	250 VU	5 min	<= 900 ms	733 ms	355.02 req/s	< 3%	2.75%	< 3%	2.75%	N/A
POST /api/expenses	50 VU	5 min	<= 700 ms			< 1%		< 1%		
POST /api/expenses	100 VU	5 min	<= 1000 ms			< 2%		< 2%		
POST /api/expenses	250 VU	5 min	<= 1800 ms			< 5%		< 5%		
GET /api/dashboard	50 VU	5 min	<= 900 ms			< 2%		< 2%		
GET /api/dashboard	100 VU	5 min	<= 1200 ms			< 3%		< 3%		
GET /api/dashboard	250 VU	5 min	<= 1800 ms			< 5%		< 5%		
POST /api/auth/login	50 VU	3 min	<= 700 ms	1117 ms	101.94 req/s	< 1%	0%	< 1%	0.00%	5
POST /api/auth/login	100 VU	3 min	<= 1000 ms	2216 ms	104.29 req/s	< 2%	0%	< 2%	0%	5
POST /api/auth/login	250 VU	3 min	<= 1800 ms	5929 ms	103.79 req/s	< 5%	0%	< 5%	0%	5
POST /api/auth/refresh	50 VU	3 min	<= 700 ms	283 ms	967.14 req/s	< 1%	0%	< 1%	0.03%	3
POST /api/auth/refresh	100 VU	3 min	<= 1000 ms	373 ms	992.72 req/s	< 2%	0%	< 2%	0.01%	3
POST /api/auth/refresh	250 VU	3 min	<= 1800 ms	736 ms	986.15 req/s	< 5%	0%	< 5%	0%	3
GET /api/health	50 VU	2 min	<= 100ms			< 0.1%		< 0.1%		
GET /api/health	100 VU	2 min	<= 120 ms			< 0.1%		< 0.1%		
GET /api/health	250 VU	2 min	<= 200 ms			< 0.5%		< 0.5%		

At the beginning of our testing phase for the scalability of Budgetwise, benchmarks for 95th percentile (latency), throughput, and error rates were created. High load was anticipated with 50-250 virtual users sending hundreds of concurrent requests, so the benchmarks were estimated to give some breathing room while anticipating degradation at higher loads. Early tests showed high throughput, low latency, and low error rate was possible in areas such as the dashboard. However, we experienced issues in our login and expenses pages. Our scalability changes took some of the load off of those areas, and now we can see post-change metrics with improvements in all areas.

5.3 AI model performance



We made an effort to limit token usage by engineering the prompts in such a way that less tokens are used per prompt. The graph above shows a before and after of when the changes to the prompts were made. This helps to lower overall costs in the future, increases efficiency of the program, and limits the environmental impact of computing the AI features.

6 Lessons learned

6.1 Insights gained

AI engineering and traditional coding essentially contain the same steps: plan, implement, debug. However, the way in which these steps are completed are different for each. In traditional coding, a programmer will plan by creating a pseudocode or UML diagrams, implement by writing code, and then debug/test manually. In AI engineering, a developer will plan by giving the AI agent a prompt and asking it to create a plan, tell the AI to implement its plan, and then debug alongside the AI.

6.2 Lessons from challenges

6.2.1 AI Limitations

Given the risky nature of using agentic AI as the primary method of programming, we created guardrails within our project specifications in order to ensure that the AI was working properly and would not hallucinate. We also ensured that the AI carefully explained every step it was taking by making it plan out its actions before executing the agentic implementation. This limited the number of hallucinations, bugs, and other mistakes introduced by AI.

6.2.2 Collaboration and Teamwork

Collaborating on a complex system over a long period of time introduces multiple challenges with communication, time management, and programming itself. To overcome these challenges, we developed a series of agreed upon systems and platforms to communicate through. For example, every week we had a meeting with the entire team, the TA, and sometimes the professor, where we summarized our work that week, asked clarifying questions, got feedback, and created a to-do list for the following week. This ensured that the team stayed consistent and informed on all changes. We also agreed upon Discord [46] as our main method of communication to limit the number of platforms we had to manage. In addition, we used Jira [47] to manage tasks and to-dos so that we could easily see who was working on what and what progress they have made. Finally, we heavily leaned upon GitHub to manage multiple different features/branches within our code to ensure that all components worked together seamlessly.

6.2.3 Funding Limitations

As this is a senior-level design class, we did not have any funding for this project. This introduced limitations, especially when working with AI models. However, we found several free tools that helped circumvent these funding limitations. We used Cursor [43] and GitHub Copilot Pro [45] to code our program using agentic AI, both of which have free options available only to college students. We also utilized Groq [8], which has generous rate and token limits available on its free tier for many different AI models when calling them through an API. We also made sure to deploy using Render [13], which has a free Hobby tier available to everyone. Utilizing these free tools helped ensure that we did not incur any costs during this project.

6.3 Skills developed

Before this project the team's general understanding of how to utilize AI was just to prompt, review, edit, and repeat. During this project the team gained a better understanding of how to utilize AI in a more efficient way. This starts with creating a project specification document with the help of the AI agent(s)

based on the project proposal. This project specification document acts as a source of truth for the AI and helps to reduce any confusion or misunderstanding for it. It is also a living document that can be changed throughout the project and the model can adapt as the document is updated. Using this document eliminates a lot of the inconsistency that comes with using AI as it has a better understanding of what the overall project is intended to look like when working on smaller portions. This has made the team's ability to utilize AI much better.

Another important skill is being able to work in a team to create a commercial grade product. The team was already familiar with the agile method and using sprints to complete a project but this gave the team an opportunity to expand these skills. Most of the team had also never had to work on a project while doing weekly update meetings with a supervisor. The team was able to learn what it was like to adapt based on the supervisors feedback and build the necessary skills to succeed in an agile development environment.

6.4 Individual reflection

Sophia Chau

During this project, I gained insights into coding a full-stack project using agentic AI tools. As the technology continues to develop, I believe this experience will help me as a burgeoning software engineer starting her career. I faced multiple challenges during this project, including using tools I had never used, working with agentic AI for the first time, and collaborating with other people when building a complex system. However, I overcame these challenges, and we have delivered what I believe to be a good product. Furthermore, I gained experience in deployment and collaboration through GitHub (e.g., merging branches) that I believe will help me in my career. Overall, I had a positive experience on this project, and I look forward to applying the skills I learned to real-world situations when I enter the industry.

Jonathan Gruber

This project presented me with many firsts: my first substantial exposure to web programming, my first time working upon an application similar to those found in the “real world”, and my first substantial exposure to various technologies such as JavaScript, TypeScript [3], Node.js [48], Docker [11], PostgreSQL [7], Postfix [15], and Render [13].

Though I was initially apprehensive about the project given its scale and the type of programming that I would be expected to do, because I was primarily a backend developer for the project, which I much prefer to being a frontend developer, the work was not too dissimilar to that of prior projects upon which I have worked. Moreover, though I did have to learn to use various technologies for this project, their documentation was generally sufficient for me to gain a basic understanding of them for use in this project. Aside from the challenge of learning to use those various technologies, two particular challenges, or, rather, annoyances, were the inability to connect the self-hosted mail server running upon my machine to external mail servers and the inability to deploy a self-hosted mail server in Render (at least with Render’s free tier). The (imperfect) solution to both these challenges was to relay the application’s email messages through an external mailbox (e.g. a throwaway Gmail account), which did allow properly delivering the application’s email messages to external mail servers but did not allow sending those messages from a custom email domain.

One takeaway of mine from this project is that, given that the project was my first substantial exposure to web programming, should I pursue web programming further, I still have much to learn about it. Another takeaway of mine from the project is greater knowledge and appreciation for the various protocols

involved in email; though I had exposure to these protocols from having previously taken a course on computer networking, I learned much more about them from using them for the project. Related to this, another takeaway of mine from the project is an appreciation for Postfix, the software package that we used for the self-hosted mail server; I was pleased to find a free and open-source mail server that follows the UNIX tradition and is robust, versatile, and not too difficult to administer (at least for our purposes in the project).

Haru Kimura

This project gave me a much better understanding of what it is actually like to work on a larger full-stack application with multiple people contributing at the same time. Most of my work focused on the frontend side of the project, user account features, and integrating frontend pages with backend functionality. One of the main features I worked on was the forgot password and reset password flow. Since we built a lot of that functionality ourselves instead of relying fully on external services, I learned a lot about authentication, validation, error handling, and how frontend and backend systems connect together.

One of the biggest challenges for me during this project was debugging integration issues. There were many situations where a feature looked correct on the frontend but would fail because of backend responses, routing issues, expired tokens, or environment variable problems. Working through those issues taught me how important testing and communication are when multiple systems depend on each other. Overall, this project helped me become more comfortable working collaboratively, troubleshooting problems, and contributing to a project that felt much closer to a real-world application than most projects I have worked on before.

Casandra Marquez

This project was a great opportunity for me to expand my technical skill set and gain experience in a more backend focused role. I initially expected to split my time evenly between frontend and backend development, but I ended up working heavily with Docker [11], Render [13], and the full deployment process from development to production. Since most of my previous experience had been isolated to either frontend or backend work, learning deployment infrastructure and containerization helped me better understand the full stack development process as a whole. Although the learning curve was steep at first, becoming comfortable with these tools was an extremely rewarding experience.

Additionally, serving as team lead for two weeks helped me grow professionally in ways I did not expect. I was responsible for organizing project tasks into tickets, assigning work across the team, reviewing code contributions, and merging everyone's work together into a cohesive product. Prior to this experience, I had not considered myself someone interested in leadership roles, but it gave me a new perspective on the value of technical leadership and collaboration. Overall, this project strengthened both my technical abilities and my confidence in taking on greater responsibility within a team environment.

Marcelo Mejia

In the process of this project, I was able to contribute to a workflow that I had not yet experienced in regular UTD classes, and in doing so pushed myself out of my comfort zone. For example, the team was larger and more role-distributed than in my earlier projects. We had to plan API structures, as well as the responsibility that each team member carried over certain features.

One of the biggest challenges I faced was managing repo conflicts during my time as team lead. Unlike the times working on my own, version control was less volatile because I had a consistent idea of the state

of the project and the commits I wanted to stage. In comparison, having to compound everyone's contributions into a single working version took time and effort in a careful manner, mostly due to the fact that I didn't want any staged features breaking under my lead.

My favorite part about working on this project was adding quality of life changes to the app. Towards the midpoint of the project timeline, I started to notice ways in which our features could be improved to make things more intuitive and convenient for the user. Adding a search bar for the expenses page, allowing the user to change between light/dark mode, and adjusting the size of UI elements for readability are some examples of changes that made everything feel like an industry-level application that took their user's needs into account.

All things considered, I am very satisfied with the work my team and I were able to put out and am extremely grateful for the opportunity this project has presented in honing my technical and communication skills.

Jonathan Strong

This project allowed me to better understand the agile development process and see what it looks like at a high level. I was also able to gain a better understanding of what a commercial grade product really looks like and what all needs to go into creating that. Both of these skills have helped to prepare me for the future as agile has been and will most likely continue to be a popular/common way to develop products. Another skill that I learned was how to use AI effectively which could be useful if I end up in a position that requires me to use AI in the way I did during this project. I also learned how to use some new tools which was a challenge initially, but now has made me a more experienced individual.

7 Conclusion & future work

7.1 Summary of key accomplishments

Overall the team was successful in completing many of the must-have and nice-to-have goals that were originally set. Our final product contains categorized expense and income tracking and the ability to set monthly budgets with category customization. It features four pages including a dashboard where the AI suggestions and expense allocation live, an expense page that allows for managing of expenses and incomes, a calendar where the user can view expenses and incomes on a month to month basis, and a budget creator page where the user can either create their own budget or use AI suggestions to create a budget. This web application is available to anyone, not just students, and is a responsive web interface built with React [1] and backend APIs using Next.js [2].

The only feature that is not in the final version that was originally meant to be there is goal tracking for student loans, emergency funds, and investing. The user is able to allocate their budget for these things however there is no way to set a dedicated goal amount.

7.2 Future enhancements

If given more time, then the team would have liked to add full bank integrations with automated transaction syncing using a paid tier of Plaid [9] as opposed to Plaid Sandbox [18]. Another potential future enhancement is more advanced AI-driven insights, alerts, and recommendations, together with using that information to create long term financial projections beyond a single month. Yet another potential future enhancement is, when the application is deployed in the cloud, to self-host the mail server used by the application to send email messages to users; this is already the case when deploying locally via Docker Compose [12], but the team unfortunately was not able to accomplish the same when deploying in the cloud. In any case, though, it would most likely be the team's first priority to finish the goal-tracking feature missing from the final version presented. If the web application were to grow in the number of users, then more scalability would need to be done, most likely through upgrading to paid tiers, where applicable, of the third-party resources used to develop the application, as well as dealing with any other scalability issues as they arise.

References

- [1] Meta Platforms Inc. and React contributors, “React,” *react.dev*, 2026. <https://react.dev/> (accessed May 04, 2026).
- [2] Vercel Inc., “Next.js by Vercel - The React Framework,” *nextjs.org*, 2026. <https://nextjs.org/> (accessed May 04, 2026).
- [3] Microsoft Corporation and TypeScript contributors, “TypeScript: JavaScript With Syntax For Types,” *typescriptlang.org*, 2026. <https://www.typescriptlang.org/> (accessed May 04, 2026).
- [4] Tailwind Labs Inc. and Tailwind CSS contributors, “Tailwind CSS - Rapidly build modern websites without ever leaving your HTML,” *tailwindcss.com*, 2026. <https://tailwindcss.com/> (accessed May 04, 2026).
- [5] Express contributors, “Express - Node.js web application framework,” *expressjs.com*, 2010. <https://expressjs.com/> (accessed May 04, 2026).
- [6] Prisma and Prisma contributors, “Prisma | Instant Postgres plus an ORM for simpler db workflows,” *prisma.io*, 2026. <https://www.prisma.io/> (accessed May 04, 2026).
- [7] The PostgreSQL Global Development Group and PostgreSQL contributors, “PostgreSQL: The world’s most advanced open source database,” *postgresql.org*, 2026. <https://www.postgresql.org/> (accessed May 04, 2026).
- [8] Groq Inc., “Groq is fast, low cost inference,” *groq.com*, 2026. <https://groq.com/> (accessed May 04, 2026).
- [9] Plaid Inc., “Plaid: Enabling all companies to build fintech solutions,” *plaid.com*, 2026. <https://plaid.com/> (accessed May 04, 2026).
- [10] Google DeepMind, “Gemini 3 — Google DeepMind,” *deepmind.google*, 2026. <https://deepmind.google/models/gemini/> (accessed May 04, 2026).
- [11] Docker Inc., “Docker: Accelerated Container Application Development,” *docker.com*, 2026. <https://www.docker.com/> (accessed May 04, 2026).
- [12] Docker Inc., “Docker Compose | Docker Docs,” *docs.docker.com*, 2026. <https://docs.docker.com/compose/> (accessed May 04, 2026).
- [13] Render, “Render | The cloud for builders,” *render.com*, 2026. <https://render.com/> (accessed May 04, 2026).
- [14] Internet Assigned Numbers Authority, “Service Name and Transport Protocol Port Number Registry,” *iana.org*, May 04, 2026. <https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml> (accessed May 04, 2026).
- [15] W. Venema and Postfix contributors, “The Postfix Home Page,” *postfix.org*, 2026. <https://www.postfix.org/> (accessed May 04, 2026).
- [16] SMTPedia, “The Complete Gmail Email Settings: POP3, IMAP, and SMTP,” *smtpedia.com*, Apr. 26, 2026. <https://smtpedia.com/gmail-email-settings-pop3-imap-smtp/> (accessed May 04, 2026).
- [17] Plaid Inc., “Link - Overview | Plaid Docs,” *plaid.com*, 2026. <https://plaid.com/docs/link/> (accessed May 04, 2026).
- [18] Plaid Inc., “Sandbox - Overview | Plaid Docs,” *plaid.com*, 2026. <https://plaid.com/docs/sandbox/> (accessed May 04, 2026).
- [19] Plaid Inc., “API - Overview | Plaid Docs,” *plaid.com*, 2026. <https://plaid.com/docs/api/> (accessed May 04, 2026).
- [20] Broadcom Inc. and Spring Boot contributors, “Spring Boot,” *spring.io*, 2026. <https://spring.io/projects/spring-boot> (accessed May 04, 2026).
- [21] D. R. Hipp and SQLite contributors, “SQLite Home Page,” *sqlite.org*, Apr. 23, 2026. <https://sqlite.org/> (accessed May 04, 2026).
- [22] P. Hazel, M. Cardwell, N. Metheringham, and Exim contributors, “Exim Internet Mailer,” *exim.org*, 2026. <https://www.exim.org/> (accessed May 04, 2026).

- [23] G. Chehade, E. Faurot, and OpenSMTPD contributors, “OpenSMTPD,” *opensmtpd.org*, 2026. <https://www.opensmtpd.org/> (accessed May 04, 2026).
- [24] M. Collina and Autocannon contributors, “mcollina/autocannon: fast HTTP/1.1 benchmarking tool written in Node.js,” *github.com*, 2026. <https://github.com/mcollina/autocannon> (accessed May 09, 2026).
- [25] A. Baldwin, C. Byström, J. Heyman, L. Holmberg, and Locust contributors, “Locust - A modern load testing framework,” *locust.io*, 2026. <https://locust.io/> (accessed May 10, 2026).
- [26] I. Sysoev, Nginx Inc., and Nginx contributors, “nginx,” *nginx.org*, 2026. <https://nginx.org/> (accessed May 09, 2026).
- [27] Okta Inc., “auth0/node-jsonwebtoken: JsonWebToken implementation for node.js <http://self-issued.info/docs/draft-ietf-oauth-json-web-token.html>,” *github.com*, 2025. <https://github.com/auth0/node-jsonwebtoken> (accessed May 04, 2026).
- [28] Recharts Group and Recharts contributors, “Recharts,” *recharts.github.io*, 2026. <https://recharts.github.io/> (accessed May 04, 2026).
- [29] Lucide contributors, “Lucide,” *lucide.dev*, 2026. <https://lucide.dev/> (accessed May 04, 2026).
- [30] Plaid Inc. and react-plaid-link contributors, “plaid/react-plaid-link: React bindings for Plaid Link,” *github.com*, 2025. <https://github.com/plaid/react-plaid-link> (accessed May 04, 2026).
- [31] WorkOS Inc. and Radix UI contributors, “Radix UI,” *radix-ui.com*, 2026. <https://www.radix-ui.com/> (accessed May 04, 2026).
- [32] Material UI SAS, “MUI: The React component library you always wanted,” *mui.com*, 2026. <https://mui.com/> (accessed May 04, 2026).
- [33] B. Luo and React Hook Form contributors, “React Hook Form - performant, flexible and extensible form library,” *react-hook-form.com*, 2026. <https://react-hook-form.com/> (accessed May 04, 2026).
- [34] Colin McDonnell and Zod contributors, “Intro | Zod,” *zod.dev*, 2026. <https://zod.dev/> (accessed May 04, 2026).
- [35] D. Wirtz and bcrypt.js contributors, “dcodeIO/bcrypt.js: Optimized bcrypt in JavaScript with zero dependencies, with TypeScript support,” *github.com*, 2025. <https://github.com/dcodeIO/bcrypt.js> (accessed May 04, 2026).
- [36] A. Reinman and Nodemailer contributors, “Nodemailer,” *nodemailer.com*, 2026. <https://nodemailer.com/> (accessed May 04, 2026).
- [37] Dotenvx LLC, “Dotenv | Secrets for developers,” *dotenv.org*, 2025. <https://www.dotenv.org/> (accessed May 04, 2026).
- [38] Express contributors, “expressjs/cors: Node.js CORS middleware,” *github.com*, 2026. <https://github.com/expressjs/cors> (accessed May 04, 2026).
- [39] E. Hahn, A. Baldwin, and Helmet contributors, “Helmet.js,” *helmetjs.github.io*, 2025. <https://helmetjs.github.io/> (accessed May 04, 2026).
- [40] Express contributors, “expressjs/morgan: HTTP request logger middleware for node.js,” *github.com*, 2025. <https://github.com/expressjs/morgan> (accessed May 04, 2026).
- [41] The Linux Foundation, “OpenAPI Initiative – The OpenAPI Initiative provides an open source, technical community, within which industry participants may easily contribute to building a vendor-neutral, portable and an open specification for providing technical metadata for REST APIs – the ‘OpenAPI Specification’ (OAS),” *openapis.org*, 2025. <https://www.openapis.org/> (accessed May 04, 2026).
- [42] Figma Inc., “Figma: The Collaborative Interface Design Tool,” *figma.com*, 2026. <https://www.figma.com/> (accessed May 04, 2026).
- [43] Anysphere Inc., “Cursor: The best way to code with AI,” *cursor.com*, 2026. <https://cursor.com> (accessed May 04, 2026).
- [44] Microsoft Corporation and Visual Studio Code contributors, “Visual Studio Code - The open source AI code editor | Your home for multi-agent development,” *code.visualstudio.com*, 2026. <https://code.visualstudio.com/> (accessed May 04, 2026).

- [45] GitHub Inc. and OpenAI Global LLC, “GitHub Copilot · Your AI pair programmer,” *github.com*, 2026. <https://github.com/features/copilot/> (accessed May 04, 2026).
- [46] Discord Inc., “Discord - Group Chat That’s All Fun & Games,” *discord.com*, 2026. <https://discord.com/> (accessed May 09, 2026).
- [47] Atlassian Corporation, “Jira | Project Management for the AI Era | Atlassian,” *atlassian.com*, 2026. <https://www.atlassian.com/software/jira> (accessed May 09, 2026).
- [48] R. Dahl, OpenJS Foundation, and Node.js contributors, “Node.js — Run JavaScript Everywhere,” *nodejs.org*, 2026. <https://nodejs.org/> (accessed May 10, 2026).

Acknowledgements

We would like to thank our professor, Dr. Sridhar Alagar, for his guidance and support throughout the project. We would also like to thank the TA, Adarsh Gella, for his frequent advice and suggestions. In addition, we would like to thank Dr. Miguel Razo for organizing CS 4485 this semester. Finally, we would like to thank Cursor and GitHub for offering free tools to students that made this project possible.